

XLSForm2Word

User Guide

Desktop tool for converting ODK, SurveyCTO, and KoboToolbox XLSForms into Word questionnaire documentation.

Version 1.2.1 | Arman Mahmud | https://www.arman-mahmud.com

XLSForm2Word turns an ODK, SurveyCTO, or KoboToolbox XLSForm into a clean, printable Microsoft Word questionnaire. It is designed for research assistants, survey programmers, M&E specialists, data analysts, and researchers who need documentation without installing Python or using an internet connection.

What you need

- Windows 10 or Windows 11
- Microsoft Word or another application that can open .docx files
- An XLSForm saved as .xlsx or .xls
- No administrator rights, Python installation, or internet connection

Start the application

1. Extract the XLSForm2Word ZIP file to a folder.
2. Double-click XLSForm2Word.exe.
3. Keep the application folder together. The settings file, logs folder, assets, and README.pdf belong beside the executable.

The application is portable. You can place the folder on a USB drive or a shared drive, but make sure you have permission to write to that location so settings and logs can be saved.

Convert a form in six steps

1. Choose the platform: ODK, SurveyCTO, or KoboToolbox. ODK is selected by default.
2. Click Browse XLSForm and select your workbook, or drag and drop the file anywhere onto the application window.
3. Review the detected languages. XLSForm2Word finds every column that begins with label:: or label:, such as label::English, label:English, or label::Bangla. Tick one or more languages.
4. Click Browse beside Output folder and choose where the Word documents should be saved.
5. Tick the table columns you want. At least one column must remain selected.
6. Click Generate Word Documents.

The progress area shows each stage: Ready, Loading form, Generating documents, and Completed. A message appears when the documents are ready, with an option to open the output folder.

Platform differences

ODK forms normally use the name column for choice values and the relevant column for relevance expressions.

SurveyCTO forms normally use the value column for choice values and the relevance column for relevance expressions.

KoboToolbox forms follow the same conventions as ODK, using the name column for choice values and the relevant column for relevance expressions.

XLSForm2Word selects the correct columns after you choose the platform. If a preferred column is not present, it tries the equivalent column and records the fallback in the log.

Languages and file names

The application creates a separate Word document for each selected language. File names follow this pattern:

QuestionnaireName_Language_word.docx

For example, a form named Household Survey with English, Bangla, and Arabic selected produces three files:

- Household Survey_English_word.docx
- Household Survey_Bangla_word.docx
- Household Survey_Arabic_word.docx

The questionnaire name comes from the form_title or form_name in the settings sheet. If those values are missing, the Excel file name is used.

What appears in the Word document

Each document contains:

- The questionnaire name and language at the top
- The generation date
- A Table Grid with the columns you selected: Variable, Question, Choices, Constraint, Relevance, and Calculation
- Choice lists formatted as [value] label, one choice per line
- A footer with the application version, developer, and a clickable website link

The document uses A4 portrait paper, the requested margins, Calibri Light 9 point text, bold table headers that repeat on every page, and italic green variable names.

Question and choice handling

Choice questions display their choices from the choices sheet. The choice value comes from name for ODK and KoboToolbox, and value for SurveyCTO. The choice label comes from the matching label:: language column.

Common non-choice types are shown with a clear placeholder when a label is not available, including Enumerator Note, Text, Date, Number, Latitude/Longitude/Altitude, and Calculated field. Structural rows such as begin_group and end_repeat, and system-generated rows such as today and deviceid, are omitted from the table.

Settings

Your platform, output folder, selected columns, and window size are saved automatically in settings.json beside the executable and restored the next time you start the application.

Use File > Settings to see the settings and log locations. You can reset preferences from this window. Resetting preferences does not delete generated documents.

Logs and troubleshooting

The application writes logs to the logs folder beside the executable. The log records startup, version, selected file, platform, detected languages, generated files, errors, and completion status.

If a log file cannot be written beside the executable, the application uses a temporary folder for logs. Open Help > Open Log File to find the active log.

The file cannot be read: Close the workbook in Excel, check that it is not corrupted, and make sure it contains both the survey and choices sheets. Try saving it again as .xlsx.

No languages are detected: Add at least one column such as label::English or label:English to the survey sheet.

The Word document cannot be saved: Close the document if it is already open in Word, or choose a folder you have permission to write to.

Choices are missing: Check that the choices sheet has list_name, the correct value column (name for ODK and KoboToolbox, value for SurveyCTO), and a matching label:: or label: language column.

A column disappears: At least one table column must remain selected, so the application restores the last column if you try to clear all of them.

Privacy

The application works completely offline. It reads the selected Excel workbook and writes Word files to the output folder you choose. It does not upload forms, labels, or generated documents anywhere.

Building from source

This section is for developers who want to rebuild the portable application. The recommended build environment is Windows 10 or Windows 11 with Python 3.12 or newer.

1. Create and activate a virtual environment.
2. Install the development requirements with `python -m pip install -r requirements-dev.txt`.
3. Start the application from source with `python app/main.py`.

4. Run the automated tests with `python -m pytest -q`.
5. Build the portable folder and ZIP with `python build.py`.

The build script regenerates the user guide PDF, runs the tests, creates the Windows version metadata, packages the application with PyInstaller, and writes the final files under `dist/`. Use `python build.py --skip-tests` only when you have already run the test suite.

Support and updates

Developer: Arman Mahmud

Website: https://www.arman-mahmud.com

For issues, include the log file and a description of the form. Do not send confidential survey data unless your organization permits it.